Guía Maestra de Arquitectura, Docker Swarm y Alta Disponibilidad

Plataforma Web y Sistema de Información Geográfica (SIG) — Cementerio General de Los Ángeles

Licitación Mercado Público: 2411-8-LE25 | **Autores:** Bryan Ahumada & Johan Muñoz | **Entorno:** Docker Swarm (Alta Disponibilidad Nativa)

---

1. Descripción General del Proyecto

El presente proyecto moderniza el sistema de catastro y gestión funeraria del **Cementerio General de la Comuna de Los Ángeles** (Región del Biobío, Chile), situado en Camino San Antonio s/n / Av. Gabriela Mistral (Coordenadas: -37.4732, -72.3229). Integra una cartografía digital interactiva de los 5 patios topográficos sobre el polígono real del recinto, catastro histórico de mausoleos emblemáticos y un motor de búsqueda multifiltro de difuntos.

Stack Tecnológico Principal:

- **Frontend:** React 18, Vite, Leaflet, Google Maps, Lucide Icons, Axios, Nginx.
- **Backend:** Python 3.10, Flask, Flask-SQLAlchemy, Flask-JWT-Extended, Gunicorn.
- **Capa de Datos:** PostgreSQL 16 + extensión geoespacial PostGIS 3.4 (Master y Réplica).
- **Balanceo & Ingress:** HAProxy 2.8 L7 (Web) y L4 TCP (PostgreSQL).
- **Orquestación:** Docker Swarm con Ingress Routing Mesh (Kernel IPVS).

2. Evolución Arquitectónica: Unidad 1 vs Unidad 2

Aspecto Técnico	Unidad 1 (Nativo GCP)	Unidad 2 (Docker Swarm)
Infraestructura	6 VMs Debian 13 en Compute Engine	Clúster Swarm multi-servicio replicado
Aislamiento	Demonios nativos systemd	Contenedores Docker multi-etapa ligeros
Redes	3 VPCs con IPs fijas (10.0.x.x)	Red virtual Overlay (cementerio_net)
Balanceo Web	HAProxy nativo en VM 1 (SPOF)	Servicio replicado (2 réplicas) en Swarm VIP
Balanceo BD	Clúster Patroni + etcd (:8008)	HAProxy TCP con failover automático ('backup')
Tolerancia a fallos	Manual / Failover por Patroni en VMs	Automática en todas las capas + Self-Healing

3. Diagnóstico y Modificaciones Realizadas Paso a Paso

Al recibir el proyecto configurado para VMs nativas, se detectaron incompatibilidades clave que fueron corregidas:

1. **Aislamiento con .dockerignore:** Se crearon archivos de exclusión en la raíz, frontend y backend para prevenir que node_modules/ y .venv/ locales saturaran las imágenes Docker.
2. **Inicialización de BD (01-init-postgis.sql):** Se automatizó la creación de app_user con contraseña AppPassword123! y permisos totales en cementerio_db.

3. HAProxy DB (swarm/haproxy-db.cfg): Se reemplazó el chequeo de Patroni (:8008) por el chequeo PostgreSQL nativo option pgsql-check user postgres y se configuró failover hacia la réplica con la directiva backup.

4. Nginx y Backend: Nginx se ajustó para servir /usr/share/nginx/html y hacer proxy a http://app_backend:5000/api/. Flask se conectó a db_balancer:5000.

5. HAProxy Web Ingress (swarm/haproxy-web.cfg): Se vinculó con el resolver DNS interno de Docker (127.0.0.11:53) y se configuró healthcheck de extremo a extremo en GET /api/health.

4. Teoría y Funcionamiento de Alta Disponibilidad en HAProxy

La Regla Física de Redes: Si una máquina o contenedor se apaga por completo, no puede ejecutar código de redirección. Por lo tanto, la Alta Disponibilidad entre balanceadores se resuelve mediante capas de coordinación:

• **Keepalived + VRRP (IP Flotante):** En servidores físicos, dos nodos comparten una IP Virtual (VIP). Si el Master muere, el Backup asume la IP en <1s mediante Gratuitous ARP.

• **Directiva 'peers' de HAProxy:** Ambas instancias sincronizan sus tablas de sesiones activas (stick-tables) por el puerto 1024, evitando que los usuarios pierdan su login al conmutar.

• **En Google Cloud Platform (GCP):** Se sitúa un Cloud Network Load Balancer L4 de Google que vigila a las 2 VMs de HAProxy y conmuta el 100% del tráfico a la VM sana.

• **En Docker Swarm (Implementado):** Swarm utiliza el Ingress Routing Mesh (IPVS en el kernel). El puerto 8000 se asocia a una VIP interna. Si una réplica de HAProxy es destruida (docker kill), el kernel envía el tráfico inmediatamente a la otra réplica en el **mismo puerto 8000 sin caída**.

5. Matriz de Servicios, Réplicas y Puertos en Docker Swarm

Servicio	Capa / Rol	Réplicas	Puertos Expuestos	Alta Disponibilidad
`cementerio_web_balancer`	Capa 1: Ingress L7	**2 / 2**	*:8000, *:8404	Swarm VIP Routing Mesh
`cementerio_app_frontend`	Capa 2: React SPA Nginx	**2 / 2**	Interno (:80)	Balanceo L7 round-robin
`cementerio_app_backend`	Capa 2: Flask Gunicorn	**2 / 2**	Interno (:5000)	Balanceo L7 round-robin
`cementerio_db_balancer`	Capa 3: Proxy DB TCP	**1 / 1**	*:7000 (Stats)	Failover 'backup' a réplica
`cementerio_db_primary`	Capa 4: PostgreSQL Master	**1 / 1**	*:5432	Transaccional & Persistencia
`cementerio_db_replica`	Capa 4: PostgreSQL Réplica	**1 / 1**	*:5433	Lectura & Standby
`cementerio_backup_server`	Capa 5: Respaldos Cron	**1 / 1**	Volumen /backups	Dumps comprimidos .sql.gz

6. Guía de Comandos Operativos

```
# Desplegar o actualizar el clúster completo:
docker stack deploy -c docker-stack.yml cementerio

# Ver el estado de todos los servicios y réplicas (2/2):
docker service ls

# Ver tareas individuales y salud de cada contenedor:
docker stack ps cementerio

# Ver logs en vivo del balanceador web:
docker service logs -f cementerio_web_balancer

# Detener el clúster:
docker stack rm cementerio
```

7. Credenciales y URLs de Acceso

- **Plataforma Web Principal:** http://localhost:8000 (Ingress Swarm)
- **Panel Estadísticas HAProxy Web:** http://localhost:8404
- **Panel Estadísticas HAProxy DB:** http://localhost:7000
- **Usuario Administrador:** admin@losangeles.cl | **Contraseña:** Admin123!
- **Usuario Funcionario:** funcionario@losangeles.cl | **Contraseña:** Funcionario123!
- **Usuario Terreno:** terreno@losangeles.cl | **Contraseña:** Terreno123!

8. Protocolo de Demostración Universitaria (El 'Show' de HA)

Para realizar la defensa ante el profesor demostrando **tolerancia a fallos real en vivo**:

```
Paso 1: Demostrar Funcionamiento Normal
- Abre http://localhost:8000 en el navegador y muestra el mapa de patios topográficos.
- En la terminal, corre: docker service ls (muestra réplicas 2/2).

Paso 2: Matar el Balanceador Web en Vivo (Capa 1)
- Ejecuta: docker kill $(docker ps -q -f name=cementerio_web_balancer | head -n 1)
- Inmediatamente recarga http://localhost:8000 o corre: curl http://localhost:8000/api/health
- Explicación: "El sistema responde 200 OK en el MISMO puerto 8000 sin caída gracias al Routing Mesh."

Paso 3: Demostrar Auto-Recuperación (Self-Healing)
- A los 5 segundos ejecuta: docker service ps cementerio_web_balancer
- Explicación: "Docker Swarm detectó la muerte de la réplica y levantó un nuevo contenedor automáticamente."

Paso 4: Matar el Backend Flask en Vivo (Capa 2)
- Ejecuta: docker kill $(docker ps -q -f name=cementerio_app_backend | head -n 1)
- Consulta: curl -s http://localhost:8000/api/patios | jq .success
- Explicación: "La consulta a la base de datos respondió 'true' sin interrupción vía la réplica activa."
```